

Citi Bike Rebalancing Explorer

Project Assessment — Datasets, Methods, Findings & Credibility Review

Updated July 24, 2026 (v4 — corrected weather-zone geography, ground-truth spot-checking, reproducibility tooling, arrivals/departures split) · Portfolio project for NYC DOT Bike Share & Shared Mobility review · Public data only, \$0 stack

2,516

stations covered (was 2,347)

12

real contiguous months, all 4 seasons

3,794

lines of pipeline code, 15 modules

166

automated tests, all passing

Bottom line: since the first version of this report, the project closed its single biggest credibility gap — the map, the elasticities, and the demand model all now run on a real, full contiguous year (Jul 2025–Jun 2026) instead of three disconnected months, and the honest "the ML model loses" finding has gone from a single train/test split to a statistically rigorous 12-fold result. This project is credible **as a methods & rigor portfolio piece**, more so than before — the same discipline (report the loss, catch your own bugs, check your own confounds) now rests on a materially stronger dataset. It is still **not an operational tool** — the live GBFS feed isn't dense enough yet for the deferred risk model, and several smaller gaps remain. Section 8 details what to disclose before presenting; Section 9 lays out what's left.

1 — Datasets

All data is public; nothing in the pipeline or dashboard is synthetic. Four independent sources feed the project:

Source	Content	Volume / range	Access
Citi Bike trip archives (S3)	Individual trip records: start/end station, timestamps, bike type	Jul 2025–Jun 2026, all 12 months — processed one month at a time and each raw zip deleted immediately after aggregation (disk-safe; the archive is public and always re-fetchable, so nothing here is a unique copy)	Public S3 bucket, no key
Open-Meteo historical archive	Daily mean temperature (°C) and precipitation (mm)	Jul 2025–Jun 2026 , 18 real geographic zones (a fixed 6km grid over the real station footprint) — one real Open-Meteo pull per zone, resolved this round after an interim k-means version was found to underserve sparse equity-priority areas, see §4	Free API, no key
NYC Open Data — NYCHA	Public housing development locations (polygon centroids)	216 records	Socrata, dataset

Source	Content	Volume / range	Access
developments			phvi-damg
NYC Open Data — school locations	School point locations, filtered to <code>status_descriptions == "Open"</code>	1,899 records; "2019–2020" vintage — verified 2026-07-23, not assumed , still the only row-queryable, coordinate-bearing school-location dataset NYC Open Data has (see §8)	Socrata, dataset wg9x-4ke6
MTA subway entrances	Subway station entrance points	2,120 records	Socrata, dataset i9wp-a4ja
GBFS live feed	<code>station_status + station_information</code> (live bikes/docks, capacity)	Running continuously via a GitHub Actions cron; 147 real snapshots over ~10 days as of this writing — see §6	Public GBFS endpoint, no key

The 12-month archive window was chosen deliberately, not arbitrarily: the in-story date is 2026-07-22, so calendar-year 2026 isn't complete yet (verified directly against the S3 bucket — the 2026-07 archive returns 404). The rolling most-recent full year (Jul 2025–Jun 2026) is the most current complete window available, confirmed against the real bucket rather than assumed.

2 — Data Manipulation Pipeline

Quality control (fixed rules, per project standard, unchanged)

- Trips under 60 seconds or over 4 hours are dropped — a small, consistent fraction across all 12 months, same rule applied identically to every month.
- Missing start/end station id is *flagged*, not dropped (`has_valid_departure/arrival_station`) — the large majority of missing end-station ids are e-bike trips, a genuine data characteristic, not a quality defect.
- Column schema was explicitly verified consistent across all 12 months before trusting the concatenation — not assumed, since the window spans two calendar years and Citi Bike has changed its schema before.

Core transformations

- **Net flow** (`flows.py`): arrivals – departures per (station, month, day-type, hour), normalized to a per-day rate. A new companion function, `compute_daily_flow_components()`, additionally persists real per-date, per-hour arrivals/departures counts separately (not just net) to `data/daily_net_flow.parquet` — 11.78M rows, the table the demand model and elasticities refit in §4 both now read directly.
- **Full-year orchestration** (new: `build_full_year.py`): loops all 12 months — download, QC, aggregate, **delete the raw zip** — before moving to the next, so peak disk usage never exceeds roughly one month's zip. Station roster churn is tracked explicitly via a `months_present` field per station (2,516

stations have any coverage; 1,672 have all 12 months) rather than silently including or excluding partial-year stations.

- **Equity join** (`equity_join.py`): unchanged methodology, re-run against the larger roster — 302 stations near NYCHA housing, 1,537 near a school, 357 in a subway-access gap (of 2,516, up from 295/1,472/305 of 2,347).
- **Station typology** (`station_typology.py`): k-means on weekday curve shape, re-fit on the full year. $k=2$ (unchanged), but **silhouette improved from 0.2819 to 0.3854** — a full year separates the two behavioral clusters more cleanly than one month did. Commuter core: 684 stations; Residential feeder: 1,146; 686 excluded as low-signal.
- **QC bounding-box rule** (`new, qc.py`): drops any trip whose start/end coordinate falls outside a generous tri-state bounding box. Found real contamination while scoping multi-point weather below — two stations literally named "LA Metro Demo" at real Los Angeles coordinates and one at (0, 0) — all three near-zero-signal but still trivially inflating the subway-gap equity count by 3. Corrected surgically in the current artifacts rather than a full re-download (see §6).
- **Route planning** (`plan_routes.py`): unchanged — greedy nearest-neighbor, capped at 45 stops/truck, depot location still a placeholder (see §8).

Six real bugs have now been found and fixed across the project's history — the original two (dtype mixing, backtest cartesian fan-out) plus four more caught specifically while building the full-year, multi-point-weather, and weather-zone-correction work, all six caught by distrusting a suspicious number or checking real data before building on top of it, not by a planned test:

1. **Station-id dtype split** (original) — 844 of 2,347 stations had their curve values silently corrupted by inconsistent pandas type inference across CSV chunks.
2. **Backtest cartesian fan-out** (original) — a duplicate-key join produced an impossible >100% coverage figure.
3. **Typology NaN-poisoning** (full-year work) — a station with zero training rows at some hour left a real NaN cell after `.unstack()` that a later `.reindex(fill_value=0.0)` did not actually fill (that argument only fills entirely-missing columns, not NaN cells within existing ones), crashing every downstream KMeans fit against the real 11.78M-row table. Never surfaced against small unit-test fixtures where every station was active every hour.
4. **Index-alignment scoring bug** (full-year work) — a scoring function compared a merge-reset-indexed prediction Series against a still-originally-indexed actual Series via pandas' automatic index alignment, silently producing NaN for 11 of the demand model's 12 walk-forward folds while the first fold coincidentally looked correct (the two indices happened to line up by chance). Fixed by comparing plain arrays positionally instead of relying on index alignment.
5. **Corrupted station coordinates** (found while scoping multi-point weather) — two stations literally named "LA Metro Demo" at real Los Angeles coordinates and one at (0, 0), all reading as contamination in Citi Bike's own published archive rather than a bug this pipeline introduced. Near-zero signal, but trivially inflated the equity subway-gap count by 3. See §4.
6. **Weather-zone equity skew** (found by checking the multi-point-weather fix against the specific stations it was meant to serve) — the first zone design (k-means on station coordinates) looked balanced by station count but swept two of four named equity-priority areas (East Queens, Southern Brooklyn) into distant,

station-dense zones 6-8km from their real location, because k-means optimizes population balance, not geographic representativeness. Fixed with a fixed geographic grid, verified to bring all four areas within ~2-3km. See §4.

All six have dedicated regression tests. Bugs 3-6 share a lesson: unit tests built from small, tidy synthetic fixtures did not — and structurally could not — catch any of them; all four only surfaced once the pipeline ran against the real, messy full-scale data, or (bugs 5-6) once real geography was actually checked against a specific claim instead of assumed correct because the aggregate numbers looked reasonable.

3 — EDA: Main Findings

- 2,516 stations have real coverage somewhere in the 12-month window (1,672 for the full year); the roster genuinely churns — 847 stations opened or closed at some point in the window, tracked explicitly rather than hidden.
- Equity exposure, re-computed against the full roster: of 2,516 stations, **302** are within 300m of NYCHA housing, **1,537** within 300m of a school, and **357** sit in an 800m+ subway-access gap.

Resolved since the first version of this report: the schema's `seasons{}` block (winter/spring/summer/fall) now genuinely contains all four real seasons — `granularity.seasons = ["winter", "spring", "summer", "fall"]` — not spring only. Verified two ways, not just by inspection: a new dashboard test fixture exercising all four seasons, and a real end-to-end run against the actual shipped `flows.json` confirming all four seasons are genuinely selectable and correctly recolor the map from their own real data.

4 — Prediction Modeling

The project's baseline-first rule — a richer model earns its place only if it measurably beats a plain, auditable lookup table — was re-tested from the ground up this round, specifically to answer a harder question than before: once the *diagnosed cause* of the original model's failure is actually fixed, does a richer model finally win, or was that never really the whole story?

The original finding (for context)

Session 5's original single train/test split (train Feb+Apr 2026, test held-out June) found a gradient-boosted model losing to a seasonal-naive lookup, traced to June's real temperature falling entirely outside the training range — a tree-based model clamps flat past the edge of what it saw in training rather than extrapolating. That diagnosis was correct as far as it went, but rested on one comparison.

The re-test: 12-fold walk-forward across the real full year

Three tiers, compared honestly: seasonal-naive (unchanged), a new GAM-style tier (penalized splines + Ridge, sklearn-native, no new dependency — chosen specifically because splines degrade more gracefully outside their training range than a tree does), and a re-tuned gradient-boosted model wrapped in a named, visible extrapolation guard that falls back from GBM to GAM to naive as a test row's temperature moves outside the fold's own training range. Real per-date weather, real day-of-week, and station typology were all

added as features — every specific gap in the original approach, fixed at once. Validated via 12-fold leave-one-month-out walk-forward (not one split), with station typology refit independently inside each fold's training data only, so a fold's held-out month never influences its own predicted cluster label.

Tier	Mean MAE (bikes/day)	vs. naive
Seasonal-naive	1.954	—
GAM (splines)	2.186	p=0.0024 (naive significantly better)
Guarded (GBM+GAM+naive)	2.055	p=0.0122 (naive significantly better)

The honest result held, and got more rigorous, not less. Fixing the exact diagnosed problem — real daily weather, real day-of-week, typology, tuning, and a fair 12-fold test instead of one extrapolation-doomed comparison — did not flip the outcome. Seasonal-naive still wins, now with Wilcoxon significance ($p<0.05$ both comparisons) instead of a single train/test split's worth of evidence. Three genuine, reported exceptions, not cherry-picked: the GBM tier beat naive outright in three *consecutive* folds — December 2025 (1.735 vs. 1.764), January 2026 (1.704 vs. 1.740), and February 2026 (1.560 vs. 1.724) — a real winter pattern, not an isolated fluke, and still not the general pattern across the full year. (An earlier version of this report undercounted this as a single exception, February only — corrected here after a full EDA-notebook rebuild caught the other two.) A secondary finding worth naming: the extrapolation guard barely ever fires in this setup, because 11 months of training already spans nearly the whole realistic NYC annual temperature range — meaning this result is close to a fair, non-extrapolating GBM comparison, and it still lost overall. The updated, more precise interpretation: station-hour net flow is dominated by a strong periodic structure a memorized lookup table already captures nearly as well as far more complex models do — a real, now statistically defensible finding, not a workaround for a data limitation.

This result is surfaced live in the dashboard's own "Model performance" panel, which now fetches data/model_performance.json rather than hardcoding numbers in the page — the exact fix for how the previous version of this table went stale (it still showed a single Feb+Apr→Jun split from Session 7.5 long after this walk-forward superseded it). The panel hides itself entirely if that file is missing, same graceful-degradation rule as every other optional layer in the dashboard.

Elasticities (capacity, temperature, precipitation) — refit on the real full year, real geography

Updated again since multi-point weather (described just below): temperature/precipitation now reflect each station's own real geographic zone's weather, not one city-wide value. The shift is small and real, not noise — exactly what genuine spatial variation should look like once it's actually measured instead of assumed away.

	Commuter core	Residential feeder
Capacity elasticity	0.0187 [0.0138, 0.0236]	0.0139 [0.0098, 0.0181]
Temperature elasticity	0.0341 [0.0335, 0.0346]	0.0374 [0.0370, 0.0378]
Precipitation elasticity	−0.0172 [−0.0182, −0.0161]	−0.0180 [−0.0188, −0.0172]
n daily observations	230,915 (was 62,720)	385,102 (was 106,965)

- **New this round: every elasticity now ships a 95% confidence interval**, not just a point estimate — computed from the OLS coefficient covariance (temp/precip directly; capacity via the delta method, since it's a derivative of a fitted curve, not a raw coefficient). Every interval above excludes zero: all three effects are now genuinely statistically significant, not point estimates a reviewer has to take on faith.
- The underlying regression was rewired to read `daily_net_flow.parquet` directly instead of re-downloading raw trips — the module had been silently broken by Session 29's own correct behavior (raw zips deleted after aggregation), still pointed at a hardcoded 3-month set. This took real daily observations from ~90 non-contiguous days to the full ~365-day contiguous year, ~3.6–3.7x more data per group.
- **Reassuring, not a correction:** point estimates barely moved from the old 90-day version. The smaller sample was already roughly right; the full year mainly buys precision (the tight new confidence intervals), not a different answer.
- All elasticities remain explicitly labeled non-causal (cross-sectional correlation, not a within-station natural experiment) in both the code and the UI — unchanged.

Multi-point weather (new): real geographic variation, not one city-wide value

Weather had been a single fixed reference point (near Central Park) since the pipeline's early sessions — every station, regardless of borough, received identical daily temperature/precipitation. This was flagged as the clearest remaining high-leverage credibility gap once the full-year data, walk-forward validation, and elasticity confidence intervals were all in place. The first attempt (k-means clustering real station coordinates into 4 zones, the same method the pipeline uses for behavioral station typology, applied to geography instead) shipped, then was checked against the specific equity-priority areas this project frames as its purpose — and found wanting.

Found by checking against the right metric, not the convenient one: the 4 k-means zones looked reasonable by station count (588–706 per zone), but that measures population balance, not geographic representativeness — different things when the station roster itself is density-skewed toward the urban core. Checked directly against South Bronx, Upper Manhattan, East Queens, and Southern Brooklyn: the two dense areas landed within ~2–3km of their assigned zone's weather point, but East Queens (6.35km off) and Southern Brooklyn (8.27km off) were swept into distant, station-dense zones — exactly the equity-priority stations this fix was supposed to serve, underserved by it. Raising k to 8 didn't fix it (East Queens still 5.06km off) because the skew came from k-means' population-balancing objective itself, not from too few clusters. Replaced with a fixed 6km geographic grid (18 real zones, station density irrelevant to assignment by construction) — all four areas now land within 1.65–2.63km, matching or beating the previously-well-served areas.

Re-running the full walk-forward and elasticities against the corrected grid changed the underlying features but not the headline conclusion: seasonal-naïve still wins overall, and the same three winter folds (Dec 2025/Jan 2026/Feb 2026) still beat it with raw GBM. Reassuring in the same way the full-year elasticity refresh was — the finding is robust to more realistic input, not an artifact of the coarser version that preceded it. This is now the sixth real bug this project's own history has caught (\$2) — and specifically the second time a fix aimed at serving equity-priority areas was checked against those exact areas rather than a convenient proxy metric, and found to need a second pass before it actually did.

Arrivals/departures split: tried, and honestly it lost

daily_net_flow.parquet has carried separate arrivals_count/departures_count columns since the full-year rebuild, specifically so this stayed answerable later. The question: does fitting each as its own count regression (Poisson loss — statistically the right choice for a real non-negative count, unlike net flow, which mixes sign) and recombining as predicted_net = predicted_arrivals – predicted_departures beat predicting net directly?

Tier	Direct net (above)	Recombined arrivals/departures
Seasonal-naive	1.954	1.954 (identical by construction — see note below)
GAM	2.186	2.180 (negligible)
Guarded (GBM+GAM+naive)	2.055	2.096 (worse)

Baseline-first, and it lost again — reported honestly, not adjusted away. Recombining two independently-fit models compounds their individual errors rather than letting one model exploit whatever correlation exists between a station's real arrivals and departures on a given day. Each component model is fine on its own terms — GBM comfortably beats the Poisson GAM at predicting real arrivals/departures counts directly (mean MAE 1.98 vs. 3.46) — so the split itself isn't broken; recombination is just a lossier path to net than fitting net directly. (The naive tier is identical either way by construction, not by re-measurement: weighted-averaging arrivals and departures separately and subtracting is mathematically the same as weighted-averaging net directly, verified by a dedicated test rather than assumed.) Seasonal-naive still wins overall, now for the second time in this project's history against a more sophisticated challenger — GBM-vs-naive originally, above, and now split-vs-direct — a real, repeating pattern specific to this dataset, not one unlucky comparison.

5 — The Application

A Leaflet-based interactive dashboard over the now 2,516 stations, built with a strict "graceful degradation" rule: every optional layer reads its own JSON file, and simply hides its control if that file is missing.

Core dashboard

- Map with marker clustering, mode-aware coloring
- Historical / Live (GBFS) mode toggle
- Day-type, season/month, and 24h hour controls, with hour auto-play — season/month selector now verified against real 4-season, 12-month data (18 real dropdown options), not just a single-season fixture
- Station detail panel (24h flow graph; cluster type + equity context under "More info")

Investigator Mode (what-if sandbox)

- Equity threshold sliders (300m/800m defaults) — recount only, deliberately no map-wide filter/recolor (standing scope decision, unchanged)
- Fleet-size simulator (1–10 trucks) against precomputed route scenarios
- Weather scenario presets (rain/snow/ideal) + manual sliders, now driven by the tighter, CI-backed elasticities above
- Persistent diff bar vs. baseline, URL-shareable saved presets, one-click reset

- Rebalancing route layer (hides if `route.json` missing)
- Legend, and a "Model performance" disclosure now *fetches live* from `data/model_performance.json` (§4) instead of hardcoded — can't silently go stale the way it did before

Deliberate scope exclusions (by design, unchanged)

- No map-level equity filter/recolor toggle — a standing decision. Per-station equity context stays available everywhere it already appears.
- No station-relocation / new-site modeling, no OR-Tools optimal routing, no live in-browser model inference, no multi-day simulation horizons, no live weather API calls from the browser.

6 — What's Left

Item	Status & blocker
Full seasonal coverage RESOLVED	All four real seasons now shipped in <code>flows.json</code> , verified against real data (§3, §5).
Arrivals/departures split into two count-regression targets RESOLVED	Built and honestly evaluated — the split loses. Recombining independent Poisson arrivals/departures models is slightly worse at predicting net flow than fitting net directly (guarded MAE 2.096 vs. 2.055). See §4.
Dock-capacity sandbox (Investigator Phase 5)	Still blocked, same two reasons as before: (1) GBFS density — 147 real snapshots over ~10 days as of this writing, cron confirmed genuinely running continuously, but still short of the 2–3 week target; (2) a distinct risk-based (empty-minutes) elasticity model still needs to be fit once density arrives — waiting alone doesn't resolve this.
"Heat wave" weather preset	Still deliberately excluded — the elasticity fit's top training region carries sparse-grid artifact risk. Unchanged this round.
Multi-point weather RESOLVED	18 real zones via a fixed 6km geographic grid (not k-means — the k-means version shipped first, then was found to skew away from sparse equity-priority areas and replaced, see §4), weather joined per-station-zone in both the elasticities regression and the demand model.
Ground-truth spot-checking RESOLVED	<code>pipeline/spot_check.py</code> — 5 checks against facts independently verifiable outside this pipeline (real NYC geography, a real holiday, a real seasonal pattern), all passing with wide margins. See §9.
Reproducibility tooling RESOLVED	<code>pipeline/reproduce_all.py</code> — scripted, verified end-to-end reproduction path; the fast path was actually run and reproduced <code>model_performance.json</code> byte-identically. See §9.

Item	Status & blocker
Arrivals/departures split into two count regression targets RESOLVED	Built and honestly evaluated — the split loses. Recombining independent Poisson arrivals/departures models is slightly worse at predicting net flow than fitting net directly (guarded MAE 2.096 vs. 2.055). See §4/§9.
Corrupted station coordinates (new item, found and fixed this round)	3 contaminated stations RESOLVED — see §2/§4's QC callout.
Real depot data	Still open, unchanged — feasibility not yet checked.

7 — Engineering Rigor

- **166 total tests** (was 114): 152 Python (pytest) + 14 Node smoke-test scenarios covering the dashboard's inline JS — up from 103+11. New coverage since the last version of this report: the full-year data pipeline, the walk-forward demand model (including the extrapolation guard's tier-routing logic and, this round, the Poisson-loss arrivals/departures split), the rewired elasticities loaders, a real full-4-season dashboard fixture, the live model-performance panel's graceful degradation, the QC bounding-box rule, the corrected weather-grid zoning, the school-vintage-note UI fix, ground-truth spot-checking (§4/§9), and the reproducibility-tooling orchestration script (§9).
- 3,794 lines of pipeline code across 15 modules (was 3,322 / 13) — two new modules this round: `spot_check.py` (ground-truth checks, §9) and `reproduce_all.py` (reproducibility orchestration, §9).
- All six real bugs from §2 have dedicated regression tests, not one-off fixes.

8 — Credibility Assessment: Can This Be Referenced or Presented?

What makes it credible

- **The baseline-first finding was re-tested harder, not just re-asserted** — the project fixed the exact diagnosed cause of the original model's failure and re-ran a rigorous 12-fold test rather than declaring victory on the original diagnosis alone. The finding held, and is now statistically defensible, not just plausible.
- **Six real bugs, not two**, have now been caught, root-caused, fixed, and covered with regression tests — four of them specifically because the project ran its own code against real data (full-scale, or real geography checked against a specific claim) instead of stopping at "the unit tests pass" or "the design looks right."
- **Every elasticity estimate now ships its own uncertainty**, not just a point estimate a reviewer has to trust blindly.
- A stale, hardcoded results table was caught and fixed *architecturally* (fetched from a real data file with graceful degradation) rather than patched with fresher hardcoded numbers that would just go stale again next retrain.
- All data is public, sourced, and traceable; illustrative/estimated output is explicitly labeled as such throughout the code and UI.

What to disclose before presenting externally

- ~~Weather is still one city-wide daily value, not per-station or per-neighborhood.~~ **RESOLVED** — 18 real geographic zones now (fixed grid, not k-means), see §4.
- Elasticity effect sizes are small (roughly 1–4% per unit change) — now precisely estimated (real confidence intervals), but still small in absolute terms and still cross-sectional, not causal.
- The GBFS live feed is real and running continuously, but still short of the 2–3 week density bar the deferred risk model needs — "live" claims are accurate today, "enough history for risk analysis" claims are not yet.
- The route planner's depot location is still a placeholder, not real fleet-operations data.
- The school-location dataset is still Socrata-labeled "2019–2020" vintage, not current-year — **now a verified limitation, not an unaddressed one**: confirmed 2026-07-23 that no newer row-queryable, coordinate-bearing school-location dataset exists on NYC Open Data (DOE appears to have stopped publishing this specific dataset series after 2019-2020). The dashboard's station detail panel now also surfaces this vintage caveat directly wherever a school-proximity fact is shown, not just in this report.

Verdict

Meaningfully stronger than the first version of this assessment, on the same axis that already made it credible. The project didn't just add more data — it used the new data to re-test its own honest finding under harder, fairer conditions, and the finding survived with real statistical backing. That is a more

convincing signal to a technical reviewer than either a clean success story or an unexamined negative result would be. **Still not ready to present as an operational rebalancing tool** — frame it explicitly as a methods showcase and decision-support prototype, now built on a genuinely representative full year of data, with the remaining gaps in §6 and this section named up front.

9 — Path to Higher Credibility, Applicability & Trust (updated)

The original list is reproduced below with completed items marked. Eight of the original eleven recommendations are now done; the rest are unchanged and still worth pursuing in roughly the same priority order.

Quick wins

1. ~~Ship all three already-downloaded months as real seasons.~~ **DONE** — superseded by the full 12-month rebuild (§1–3), not just the original 3-month fix.
2. ~~Refresh the school-location dataset.~~ **VERIFIED** — thoroughly re-checked, not swapped: no newer row-queryable, coordinate-bearing school-location dataset exists on NYC Open Data. wg9x-4ke6 (2019–2020) remains the only option that actually works, now documented with a dated paper trail. Found and fixed a related real gap along the way: the vintage caveat was computed but never actually shown in the dashboard UI — now it is.
3. ~~Attach standard errors to the elasticity estimates.~~ **DONE** — every elasticity now ships a real 95% CI (§4).
4. **Keep local and GitHub in sync on a routine cadence** — still worth doing as a habit; local is currently a few commits behind origin's GBFS cron again as of this writing, the same class of drift flagged the first time.

Medium investment

5. ~~Multi-point weather instead of one city-wide value.~~ **DONE** — 18 real geographic zones via a fixed 6km grid, described in full in §4. (An interim k-means version shipped first and was replaced after being found to underserve sparse equity-priority areas — see §2/§4.)
6. ~~Walk-forward validation instead of one backtest pair.~~ **DONE** — 12-fold leave-one-month-out, described in full in §4.
7. **Source a real depot location for route planning** — still open, unchanged; feasibility not yet checked.
8. ~~Spot-check against independent ground truth.~~ **DONE** — pipeline/spot_check.py, 5 checks against facts verifiable independent of this pipeline (real NYC office/residential geography, a real holiday, a real seasonal pattern), all passing with wide margins, not threshold-hugging near-misses.

Larger investment

9. ~~Extend to a full contiguous year of trip data.~~ **DONE** — the single biggest lever from the original list; everything in §1–4 is downstream of this.
10. **Build the risk-based (empty/full-minutes) elasticity once GBFS density hits the 2–3 week target** — still blocked on data density (§6); genuinely new modeling work, not yet started.
11. ~~Add reproducibility tooling.~~ **DONE** — pipeline/reproduce_all.py, a scripted end-to-end path from committed data to reproduced elasticities/walk-forward results. Not just written — actually run: the fast

path (`--skip-download`) reproduced `model_performance.json` byte-identically against the committed version.

A second honest loss, in the same spirit as the first. This round tried the arrivals/departures split originally scoped as future modeling work (§6): fit arrivals and departures as two independent Poisson count regressions and recombine into net flow, rather than fitting net directly. The baseline-first rule applied the same way it did to the original GBM-vs-seasonal-naive question — and the split lost. Guarded-tier MAE went from 2.055 (direct net) to 2.096 (recombined) — worse, not better, because recombining two independently-fit models compounds their individual errors rather than letting one model exploit the real correlation between a station's arrivals and departures on a given day. This is now the second time in this project's history that adding modeling sophistication (a richer learner, then a more granular target decomposition) has lost to the simpler alternative — a real, repeating pattern specific to how strongly seasonal this dataset's station-level flow actually is, reported here exactly as it happened rather than quietly dropped from the write-up. See §4 for the full comparison table.

What not to change: the baseline-first discipline, the willingness to let a model lose and say so in the shipped UI, and the habit of labeling every number as real vs. illustrative are still the actual source of this project's credibility — and this round is direct evidence they scale twice over: the same discipline, applied to a harder re-test, produced a more rigorous version of the same honest finding rather than a different, more convenient one, and applied again to a new question, produced a second honest loss instead of a quietly-abandoned experiment.

Sources: `PROGRESS.md` (Sessions 1–41), `pipeline/*.py` (15 modules), `data/flows.json`, `data/daily_net_flow.parquet`, `data/elasticities.json`, `data/model_performance.json`, `data/gbfs_log/snapshots.csv` (147 snapshots as of this writing), `README.md`, `CLAUDE.md`, `reference/Investigator_Mode_Guideline.md`, `git log` against the GitHub Actions history, and the project's own test suite (152 Python + 14 Node tests, run 2026-07-24). All figures above were pulled directly from code, data files, and committed progress notes — none are invented.